CS61 26S Final Project

Authors: Jack, Paul, Dylan, Jaysen

Purpose + Who it Serves

Our application serves a general user who wants a central hub to manage all of their health-related stats. Instead of tracking weightlifting, cardio exercise, sleep, and personal weight across 4 different systems, our app provides one simple home for a user to measure everything they may want related to their health. Unlike the mainstream options for this sort of app, we will prioritize simplicity by serving a specific set of users: our own group. We all are tired of relying on 3rd party apps to house our data. We would like a place where we have complete control and flexibility over how our data is tracked, so we can log our activities exactly as we wish to.

Business Rules

Entities Involved:

Our app is designed to support the following entities: users, workouts, exercises, sets, cardio sessions, weight, and sleep. Users are the ones controlling how to use our app and log their workouts and health choices. At the highest level, users log their workouts, but within their workout, there are a variety of options. If it's a weightlifting workout, the user may opt to log each of their exercises, as well as how many sets they do per lift. If it's a cardio workout, the user can log their type of activity as well as the intensity, length, and duration of their workout. If it's a mix of both, they can log both a cardio session and a weightlifting lift pointing to the same workout. As well as workouts, the user will also be able to update their weight and measure their sleep each day.

Entity Relationships + Workout Rules:

- Each user can log many workouts, but each workout belongs to exactly one user
- Each workout occurs on a single date and may contain any combination of exercises (weightlifting) and cardio sessions
- A single exercise type can be in many different workouts
- One cardio session belongs to one workout
- Each exercise per workout contains one or more sets, and each set belongs to exactly one exercise
- Each set records the weight lifted, the number of reps performed, and a rate of perceived exertion value

- Each weight measurement is recorded by exactly one user on a specified date
- Each sleep entry is recorded by exactly one user and corresponds to a single night of sleep
- Users can only access and modify their own data; no user should be able to read or write another user's workouts, sets, weight, sleep, or supplemental records

Relations:

Before discussing the schema we used for these entities, we will look at the relations between them. As mentioned, at the highest level are users. Users control their workouts, weight, and sleep. A user may have many of each of these entities, many days of sleep, many weight logs, and many workouts. However, all of those can only belong to one user. Therefore, Users are 1 to Many for Sleep, Weight, and Workouts. To model this, all entities will have a FK to the PK of Users.

Within workouts, there are relationships as well. We have a three-tier hierarchy: Workout -> WorkoutExercise -> WorkoutSets, which we believe mirrors the natural model of lifting. A workout consists of one or more exercises, and each exercise is composed of one or more sets. We choose to model the Exercises table as a catalog of popular exercises to ensure naming consistency, making it easy to rename "Bench Press" to "Barbell Bench Press", for example. This model also allows us to track a specific user's progress on an exercise over a period of time. For instance, if a user Bob wanted to track his bench press progress over time, we would join WorkoutSets -> WorkoutExercise -> Workouts, filtering by UserID = 5 and ExerciseID = 5, and order by WorkoutDate.

That said, Workouts are many to many with Exercises, as one workout can contain many different exercises and one exercise (i.e., bench press) can exist in many different workouts. Thus, we introduced a junction table called WorkoutExercise, where each instance in the table represents an "exercise" associated with some workout session. To illustrate this, consider Bob's example workout:

Bob's workout on 5/19/26:

- Exercise 1: Bench Press
    - Set 1: 8 reps, 100 lbs
    - Set 2: 7 reps, 105 lbs
    - Set 3: 7 reps, 105 lbs
- Exercise 2: Squat

- Set 1: 8 reps, 150 lbs
- Set 2: 8 reps, 145 lbs
- Set 3: 7 reps, 145 lbs

Workout's Table

WorkoutID	UserID	WorkoutDate	Description	Notes
1	5 (Bob)	2026-05-19	Push/Legs	Great workout

The WorkoutExercise table will have WorkoutID as an Fk (so we know which workout this exercise belongs to) and ExerciseID as an FK (so we know which exercise is being performed). To model Bob's workout, we would create two WorkoutExercise instances as follows:

WorkoutExerciseID	WorkoutID	ExerciseID
1001	1	5 (bench press)
1002	1	12 (squat)

**5 and 12 are arbitrary for example purposes.*

One WorkoutExercise can have many sets and each set belongs to one WorkoutExercise. We would then have a sets table that stores WorkoutExerciseID as a FK, so we know which set belongs to which exercise.

SetID	WorkoutExerciseID	SetNum	Weight	Unit	Reps	RPE
1001	1001	1	100	lbs	8	9
1002	1001	2	105	lbs	7	9
1003	1001	3	105	lbs	7	10
1004	1002	1	150	lbs	8	8
1005	1002	2	145	lbs	8	9
1006	1002	3	145	lbs	7	8

Entity Characteristics:

Now that we have modeled the entities and the relations between them, we can specify the exact DB we created.

Users(UserID, Username, Email, HashedPassword, FirstName, LastName, City, State, CreatedAt, IsAdmin)
Workouts(WorkoutID, UserID*, WorkoutDate, Description, Notes)
Exercises(ExerciseID, ExerciseName)
WorkoutExercise(WorkoutExerciseID, WorkoutID*, ExerciseID*, Notes, OrderNum)
Sets(SetID, WorkoutExerciseID*, SetNum, Weight, Unit, Reps, RPE)
CardioSessions(CardioID, WorkoutID*, ActivityType, DurationMinutes, DistanceMeters, Intensity, Notes)
Bodyweight(WeightID, UserID*, RecordedDate, Weight, Unit)
Sleep(SleepID, UserID*, SleepDate, DurationMinutes)
* Indicates a foreign key

Given this, it is important that data for one user is scoped to only that user. Our permissions system does not allow for one user to accidentally access another user's data.

Changes from Initial Plan

- **Removed MuscleGroup attribute from Exercises table**
    - Reason: exercises can hit many muscle groups → having a single enum for an exercise is not useful. For the purposes of our tracking, we got rid of the attribute
- **Add Notes attribute to WorkoutExercise joining table**
    - Reason: Users may want to annotate a single exercise within a workout. Ex. "this bench felt heavy today." This doesn't belong on the whole Exercises table since its specific to a workout but we still needed somewhere to store it
- **Removed Supplements table**
    - Reason: we had originally planned to track supplements, but as we built our app we realized that it didn't really fit with the main workout logging functionality. Supplements have different timelines and don't really progress like workouts do, so we removed the feature
- **Removed ENUM from CardioSessions attribute, ActivityType**
    - Reason: There are infinite types of cardio, so having an enum didn't make sense nor was that important. We make it a VARCHAR instead
- **Changed Intensity attribute to be within {1,2,3,4,5} instead of {'low', 'medium', 'high'}**
    - Reason: More granularity and easier to sort
- **Removed AvgHeartRate attribute from CardioSessions table**

    - Reason: most people won’t have an AvgHeart rate for their workout. Easier to just remove
- **Exercises are a shared catalog (no role/status based access)**
    - Reason: we originally wanted to have a catalog that was fixed - only developers or admins could edit or add exercises. Instead, we make them not UserSpecific because maintaining a user-specific table of exercises is not actually that valuable and massively complicates how our front-end will need to handle it. Instead, Users are able to add any exercises they want, to an appended only, de-duplicated table that we keep in SQL.
- **Added Audit Log**
    - Reason: It wasn't in our original ERD, so we added it to (1) satisfy the requirement recommended in the lab specs, and (2) get automatic, app-independent logging. The database records the events itself via triggers, so nothing depends on the application code. Triggers handle it all

All of these changes were made to either simplify the functionality of our application for our users or add simplicity and coherence to our implementation of the app. For instance, we decided to remove supplement tracking from our application. Even though this is an additive feature, when thinking more about the problem itself, we realized that it really doesn’t have a clear place in a workout tracking application. Furthermore, we added a feature like Audit table logging with triggers, so that we have a history of our edits to our workout tables.

Lessons Learned

- There are lots of different choices for how to make a DB schema. In our case, because we are not operating with huge scales of data to store and transactions per second, it doesn’t really matter which one we choose (besides our spec-specified goal of getting to normalized form and for ease of querying in our actual SQL commands). But if we were to scale this up, the efficiency and speed gains from normalization would be huge.
- Our problem was a great fit for a SQL database. We are able to easily type and set a fixed schema of workouts and exercises that our users can add from their choices on the frontend. By this nature, SQL works very well to implement this structured schema. We don’t really have a need for a NoSQL database. However, if we wanted to categorize, say, the similarity of workouts, it could be interesting to embed workouts in some way within a vector database. Furthermore, if we needed to store some form of unstructured data associated with our workouts, we might also have to branch out of a pure SQL database.